

PantherFlow Clinical

Relatório Técnico — pipeline.py

Gerado em 2026-05-19

Visão Geral

A pipeline executa em background via `processar_paciente_wsl()`, orquestrando containers Docker dentro do WSL2. Suporta Single-End e Paired-End. Todos os outputs pesados são escritos diretamente em disco via redirecionamento de shell — nunca passam pela memória Python.

Etapas 0 — Validação de Índices BWA (Barreira de Segurança)

Ferramenta: Sistema de arquivos (Python)

Critérios:

- Verifica a existência dos 5 arquivos de índice BWA: `.bwt`, `.pac`, `.ann`, `.amb`, `.sa`
- Se qualquer índice estiver ausente: `FileNotFoundError` fatal — pipeline abortada antes de qualquer I/O

Etapas 0.5 — Controle de Qualidade Bruto (FastQC)

Ferramenta: `fastqc`

Critérios:

- Processa R1 (Single-End) ou R1 + R2 (Paired-End)
- Output HTML/ZIP gerado em `/processamento/`
- Falha com código não-zero: exceção fatal — pipeline abortada

Etapas 0.75 — Trimagem de Adaptadores e Qualidade (Trimmomatic)

Ferramenta: `trimmomatic PE / SE`

Parâmetro	Valor	Razão
ILLUMINACLIP	TruSeq3-PE/SE.fa:2:30:10	Remove adaptadores Illumina TruSeq3
seedMismatches	2	Tolerância a mismatches na semente
palindromeClipThreshold	30	Score mínimo para clip palindrômico (PE)
simpleClipThreshold	10	Score mínimo para clip simples
minAdapterLength	8 (PE)	Comprimento mínimo de adaptador detectado
keepBothReads	true (PE)	Mantém as duas reads mesmo se uma for descartada
LEADING	3	Remove bases no início com qualidade < 3
TRAILING	3	Remove bases no fim com qualidade < 3
SLIDINGWINDOW	4:15	Janela de 4 bp, qualidade média mínima 15
MINLEN	36	Descarta reads com menos de 36 bp após trimagem
-threads	4	Paralelismo

Outputs — PE: 4 arquivos (`_paired` e `_unpaired` para cada read) | SE: 1 arquivo (`_trimmed`)

Etapas 1 + 2 — Alinhamento + Conversão BAM (Pipeline Unificado)

Ferramentas: bwa mem -> samtools view -> samtools sort -> samtools index

Critérios:

- SAM intermediário flui via pipe dentro do container: nunca é gravado em disco (evita ~50 GB de I/O por amostra WES e reduz o tempo em ~30%)
- set -o pipefail: propaga falha de qualquer etapa do pipe
- -t 4 e -@ 4: 4 threads em todas as etapas
- -R: Read Group injetado (ID, SM, PL:ILLUMINA, LB:lib1) — obrigatório para o Mutect2 identificar a amostra pelo campo SM:
- PE: alinha R1_paired + R2_paired | SE: alinha R1_trimmed
- Referência: Homo_sapiens_assembly38.fasta (hg38)

Etapa 2.5 — Qualidade do Alinhamento (Qualimap)**Ferramenta: qualimap bamqc**

Critérios:

- -bam: usa o BAM produzido na etapa anterior
- -gff: restringe a análise ao painel BED (alvo_qualimap_6col.bed)
- -nt 4 e --java-mem-size=4G
- Métrica extraída: mean coverageData via regex no arquivo genome_results.txt

Etapa 3 — Extração de Métricas (Flagstat)**Ferramenta: samtools flagstat**

Critérios:

- capture_output=True é usado aqui de forma legítima: saída são ~10 linhas (sem risco de buffer overflow) e precisa ser parseada por regex imediatamente
- Regex '(d+) \+ \d+ in total' -> total_reads (formato X.XM)
- Regex 'mapped \([0-9.]+\)%' -> mapping_rate

Etapa 4 — Chamada de Variantes: VarScan2**Ferramentas: samtools mpileup | varscan mpileup2cns**

Parâmetro	Valor	Razão
--variants	—	Emite apenas posições variantes
--output-vcf 1	—	Formato de saída VCF
--min-var-freq	configurável (padrão 5%)	Detecta variantes subclonais
--min-coverage	configurável (padrão 100x)	Profundidade mínima (padrão GDC)
--min-reads2	5	Mínimo de reads suporte mesmo a VAF baixo
--strand-filter 0	desativado	VarScan2 é 'rede larga'; strand bias filtrado pelo FilterMutectCalls
-B (mpileup)	—	Desativa BAQ — evita falsos negativos em INDELS
-l (mpileup)	BED file	Restringe ao painel alvo

Nota: o VCF é gerado via '>' no shell Docker — stdout nunca passa pelo Python.

Etapa 4.5 — Chamada de Variantes: Mutect2 (GATK, tumor-only)**Ferramenta: gatk Mutect2**

Parâmetro	Valor	Razão
-R	hg38	Genoma de referência
--tumor-sample	UUID	Identifica a amostra tumoral pelo Read Group

Parâmetro	Valor	Razão
<code>--min-AF</code>	configurável	Alinha limiar de emissão com o VAF do VarScan2
<code>-L</code>	BED file	Restringe ao painel alvo
<code>--germline-resource</code>	af-only-gnomad.hg38.vcf.gz	Penaliza variantes com alta freq. pop. (GATK Best Practices)
<code>-pon</code>	1000g_pon.hg38.vcf.gz	Panel of Normals — filtra artefatos sistemáticos (opcional, com fallback)
<code>--f1r2-tar-gz</code>	_f1r2.tar.gz	Coleta estatísticas de orientação para o modelo F1R2

Etapa 4.5.5 — Modelo de Artefatos F1R2 (LearnReadOrientationModel)

Ferramenta: `gatk LearnReadOrientationModel`

Critérios:

- Aprende modelo probabilístico de artefatos OxoG (dano oxidativo) e FFPE a partir das estatísticas F1R2 coletadas pelo Mutect2
- Substitui o FilterByOrientationBias (descontinuado no GATK >= 4.2)
- Fallback: se falhar, FilterMutectCalls roda sem `--ob-priors` — pipeline não bloqueada

Etapa 4.55 — Estimativa de Contaminação (GetPileupSummaries + CalculateContamination)

Ferramentas: `gatk GetPileupSummaries` -> `gatk CalculateContamination`

Critérios:

- Padrão GDC NIH para análise Tumor-Only
- Usa variantes populacionais do gnomAD como âncoras para estimar a fração de contaminação cruzada entre amostras
- Resultado (`_contamination.table`) passado ao FilterMutectCalls para ajustar limiares estatísticos ao nível real de contaminação
- Fallback em 2 níveis: falha do pileup OU falha do cálculo -> FilterMutectCalls roda no modo padrão sem tabela de contaminação

Etapa 4.6 — Filtro Estatístico GATK (FilterMutectCalls)

Ferramenta: `gatk FilterMutectCalls`

Filtros aplicados:

- Strand bias
- TLOD (Tumor Log Odds)
- Artefatos de orientação de fragmento via `--ob-priors` (se disponível)
- Fração de contaminação via `--contamination-table` (se disponível)
- `--min-allele-fraction`: alinha o limiar interno do GATK com o VAF configurado — sem este parâmetro, o filtro 'low_allele_frac' descartaria variantes de baixo AF que o VarScan2 chamaria, quebrando o consenso
- Somente variantes com FILTER=PASS entram no consenso

Etapa 4.7 — Normalização Canônica (bcftools norm)

Ferramentas: `bcftools reheader` + `bcftools norm`

Problema resolvido: INDELS representados diferentemente por VarScan2 e Mutect2 (left-aligned vs right-aligned, multi-alélico vs bi-alélico) causariam falhas na interseção sem normalização prévia.

Pipeline de normalização do VarScan2 (2 passos independentes):

- Passo A — Renomeação de cromossomos (sed): VarScan2 emite '1','2'...; hg38 usa 'chr1','chr2'... O guard `/^chr/!`

torna o sed idempotente. bcftools annotate --rename-chrs não é usado pois falha (erro 255) em VCFs sem ##contig no cabeçalho.

- Passo B — Reheader + Normalização: bcftools reheader --fai injeta ##contig a partir do índice FASTA (bcftools >= 1.9); -m-any divide multi-alélicos em bi-alélicos; -f executa left-alignment dos INDELS.

Normalização Mutect2: direto via bcftools norm -m-any -f ref (sem passo A).

Etapa 5 — Consenso Multi-Caller (VarScan2 intersecao Mutect2)

Ferramenta: Python puro (parse_vcf() + escrever_consensus_vcf())

Critérios:

- Lê os VCFs normalizados com fallback para o VCF original caso a normalização tenha falhado
- _vcf_valido(): verifica existência + tamanho > 0 + presença de ao menos 1 linha não-header
- Filtros aceitos em parse_vcf(): PASS e '.' (ponto = sem filtro aplicado, comportamento padrão do VarScan2)
- Variantes com qualquer outro FILTER (germline, weak_evidence, strand_bias, etc.) são rejeitadas antes da interseção
- Interseção por tupla (CHROM, POS, REF, ALT) — identidade exata de representação
- Template do VCF de consenso: Mutect2 (headers mais completos, compatíveis com SnpEff e GATK)
- Se a interseção for vazia: _consensus.vcf não é criado — a ausência é detectada por _vcf_valido() nas etapas seguintes

Etapa 5.5 — Anotação Funcional (SnpEff)

Ferramenta: snpEff ann | **Banco:** GRCh38.99

Critérios:

- Banco GRCh38.99 verificado antes de qualquer chamada — FileNotFoundError fatal se ausente (com instrução de download no log)
- -nodownload: garante modo offline | -Xmx4g: limite de memória JVM
- Anotação aplicada a 3 VCFs separados: VarScan2, Mutect2 e Consenso
- Falha nos VCFs individuais (VarScan2/Mutect2) -> aviso não fatal; falha no Consenso -> exceção fatal

Campos extraídos do campo ANN= (pipe-delimitado):

- effect — tipo de efeito (ex: missense_variant)
- impact — HIGH, MODERATE, LOW, MODIFIER
- gene — símbolo do gene afetado (ex: TP53)
- hgvs_p — notação proteica (ex: p.Arg248Trp)

Etapa 5.6 — Anotação Clínica (ClinVar via SnpSift)

Ferramenta: SnpSift annotate | **Banco:** clinvar.vcf.gz

Campos injetados no campo INFO:

- CLNSIG — significância clínica (ex: Pathogenic, Likely_pathogenic)
- CLNDN — nome da doença associada

Critérios:

- Escreve resultado em /tmp/ primeiro; mv para /processamento/ só ocorre se o comando teve sucesso — evita arquivo vazio em caso de falha
- chmod -R 777 libera arquivos root-owned para o processo Python do host
- Falha -> aviso no log; campos ficam ausentes no laudo, pipeline continua

Etapa 5.7 — Anotação Oncológica (COSMIC via SnpSift)

Ferramenta: SnpSift annotate | **Banco:** Cosmic_GenomeScreensMutant_v103_GRCh38.vcf.gz

Campo injetado: CNT — contagem de ocorrências da variante em tumores no COSMIC

CrITÉrios:

- Entrada preferencial: `_consensus_clinvar.vcf`; fallback: `_consensus_snpeff.vcf` (se ClinVar falhou)
- Falha -> `shutil.copy2` promove o VCF anterior na cadeia como `_consensus_annotated.vcf` para não bloquear etapas seguintes

Etapa 5.8 — Anotação Populacional (gnomAD via SnpSift)

Ferramenta: `SnpSift annotate` | Banco: `af-only-gnomad.hg38.vcf.gz`

Campo injetado: AF — frequência alélica na população geral (gnomAD)

CrITÉrios:

- Entrada preferencial: `_consensus_annotated.vcf`; fallback: VCF anterior na cadeia de anotação
- Multi-alélico tratado: `AF=0.003,0.001` -> pega o primeiro valor; valores não-numéricos são descartados silenciosamente
- Falha -> mesmo mecanismo de fallback de cópia do COSMIC

Etapa 6 — Salvamento Final e Telemetria

Ferramenta: `SQLAlchemy` (banco `SQLite` local)

Campo no banco	Fonte
<code>total_reads</code>	<code>samtools flagstat</code> — regex <code>'in total'</code>
<code>mapping_rate</code>	<code>samtools flagstat</code> — regex <code>'mapped (%)'</code>
<code>mean_coverage</code>	<code>qualimap genome_results.txt</code> — regex <code>'mean coverageData'</code>
<code>variants_varscan / mutect / consensus</code>	<code>len()</code> dos sets Python de cada caller
<code>varscan_details / mutect_details</code>	Top 20 variantes por impacto (JSON, <code>SnpEff</code>)
<code>annotation_summary</code>	<code>parsear_anotacoes_snpeff()</code> no VCF final anotado
<code>time_total</code>	Cronômetro global (<code>time.time</code>)
<code>time_steps</code>	Dict de tempos por etapa (JSON)
<code>bwa_version, samtools_version, reference_version</code>	Valores fixos para rastreabilidade clínica

Mecanismos Transversais

Mecanismo	Onde se aplica	Comportamento
<code>_executar_docker()</code>	Todas as etapas Docker	Thread dedicada para leitura do PIPE — evita hang do Docker Desktop no WSL2 (bug de teardown onde o pipe stdout não fecha após o container encerrar)
<code>set -o pipefail</code>	BWA, VarScan2, normalização	Propaga returncode de falha de qualquer etapa dentro do pipe
<code>_vcf_valido()</code>	Etapas 5 a 5.8	Verifica existência, tamanho > 0 e presença de ao menos 1 linha de dados
Fallback chain	Etapas 4.5.5, 4.55, 5.6, 5.7, 5.8	Pipeline nunca bloqueada por ferramenta opcional; cada etapa tem fallback explícito
<code>atualizar_fase('failed')</code>	Bloco except global	Banco de dados nunca fica preso no status 'processing' mesmo em crash
Limpeza prévia de arquivos	Início da função principal	Remove arquivos intermediários de execuções anteriores com o mesmo UUID